Module : Sécurité des Réseaux et IoT

Filière : 2ACI INF

RAPPORT DE TRAVAUX PRATIQUES

Sécurisation d'un Broker MQTT via TLS/mTLS

Implémentation de certificats X.509 et Analyse Wireshark

Réalisé par :
Youssef FELLAH

Encadré par :
Pr. SLAITANE

Année universitaire : 2025/2026

1 Introduction et Objectifs

Ce travail pratique s'inscrit dans une démarche de sécurisation des communications au sein d'un écosystème **Internet des Objets (IoT)**. L'objectif principal est de migrer d'un modèle de communication non chiffré (port 1883) vers une architecture robuste utilisant le protocole **TLS (Transport Layer Security)** et l'authentification mutuelle **mTLS** sur le port 8883.

Les enjeux abordés sont :

- **Confidentialité :** Chiffrement des données de capteurs en transit.
- **Intégrité :** Garantie que les messages MQTT (dont le *Last Will*) ne sont pas altérés.
- **Authentification Mutuelle :** Le Broker et le Client doivent prouver leur identité respective via des certificats X.509 avant tout échange.

2 Mise en œuvre de l'Infrastructure à Clés Publiques (PKI)

Pour établir une chaîne de confiance, nous avons déployé une autorité de certification (CA) locale chargée de signer les certificats des entités.

2.1 Création de l'Autorité de Certification (Root CA)

Le certificat `ca.crt` constitue l'ancre de confiance racine installée sur tous les nœuds.

Listing 1 – Génération de la Root CA

```
# 1. G n ration de la cl  priv e RSA 2048 bits
openssl genrsa -out ca.key 2048

# 2. Cr ation du certificat racine auto-sign  (validit  1 an)
openssl req -new -x509 -key ca.key -out ca.crt -days 365 -subj "/C=MA/O=IoT
    Lab/CN=IoT Root CA"
```

2.2 Génération et Signature des Certificats d'Entité

Nous avons généré des certificats distincts pour le Broker et le Client afin de respecter le principe d'isolation des identités.

Listing 2 – Signature du certificat du Broker

```
# 1. G n ration de la cl  priv e du Broker
openssl genrsa -out broker.key 2048

# 2. Cr ation de la demande de signature (CSR)
openssl req -new -key broker.key -out broker.csr -subj "/C=MA/O=IoT Lab/CN=
   localhost"

# 3. Signature du certificat par la Root CA
openssl x509 -req -in broker.csr -CA ca.crt -CAkey ca.key -CAcreateserial -
   out broker.crt -days 365
```

Note : Le même processus a été appliqué pour générer `client.key` et `client.crt` avec le CN `sensor1`.

3 Configuration du Broker Mosquitto

Le déploiement sur Fedora 43 nécessite une configuration spécifique pour activer le port sécurisé et forcer le **mTLS**.

Listing 3 – Configuration mosquitto-tls.conf

```
# Paramtres r seau
listener 8883
protocol mqtt

# Chemins des certificats
cafile ./ca/ca.crt
certfile ./broker/broker.crt
keyfile ./broker/broker.key

# S curit  avanc e
tls_version tlsv1.2
require_certificate true
use_identity_as_username true
```

Explication des directives :

- `require_certificate true` : Active l'authentification mutuelle en obligeant le client à présenter un certificat valide signé par notre CA.
- `use_identity_as`$_{u}sernam e$: $ExtraitleCommon\ Name(CN)ducertificatclientpourl'utiliserco$

4 Architecture de l'Espace de Travail

Pour garantir une gestion rigoureuse des identités et éviter toute confusion entre les clés privées et les certificats publics, nous avons structuré le projet dans un répertoire dédié nommé `Certificate`. Cette organisation permet d'isoler les secrets de chaque entité.

FIGURE 1 – Architecture du répertoire Certificate et répartition des identités

4.1 Description des sous-répertoires

- **`/ca`** : Contient l'ancre de confiance (`ca.crt`) et la clé de signature racine (`ca.key`).
- **`/broker`** : Regroupe la clé privée du serveur et son certificat signé, utilisés dans la configuration Mosquitto.
- **`/client`** : Stocke l'identité numérique du capteur nécessaire pour l'authentification mutuelle (mTLS).

5 Développement du Client Sécurisé

Le capteur IoT est simulé par un script Python utilisant la bibliothèque `paho-mqtt`. Le script intègre la gestion du tunnel TLS et le mécanisme de *Last Will and Testament*.

5.1 Code Source du Capteur (`secure_sensor.py`)

Le listing suivant présente l'implémentation finale adaptée à notre structure de fichiers.

Listing 4 – Implémentation du client avec support mTLS et LWT

```python
import paho.mqtt.client as mqtt
from paho.mqtt.enums import CallbackAPIVersion
import ssl
import time

# Chemins relatifs pointant vers le dossier Certificate
CA_CERT = "./Certificate/ca/ca.crt"
CLIENT_CERT = "./Certificate/client/client.crt"
CLIENT_KEY = "./Certificate/client/client.key"

client = mqtt.Client(callback_api_version=CallbackAPIVersion.VERSION2,
    client_id="sensor_1")

# Configuration du Last Will and Testament (LWT)
client.will_set(topic="status/sensor_1", payload="Offline unexpectedly",
    qos=1, retain=True)

# Chargement de la configuration TLS
client.tls_set(
    ca_certs=CA_CERT,
    certfile=CLIENT_CERT,
    keyfile=CLIENT_KEY,
    cert_reqs=ssl.CERT_REQUIRED,
    tls_version=ssl.PROTOCOL_TLSv1_2
)

client.connect("localhost", 8883)
# ... Publication de donnees ...
```

5.2 Analyse du Last Will and Testament (LWT)

Conformément aux exigences du TP (Question 6), le message de "testament" est défini lors de la phase de connexion initiale.

- **Sécurité :** Comme le message LWT est transmis au Broker *après* l'établissement du Handshake TLS, il bénéficie du chiffrement de la session.
- **Confidentialité :** Le contenu du message (statut de déconnexion) est ainsi protégé contre l'interception sur le réseau.

6 Analyse de la Capture Réseau et Évaluation

La validation de la sécurisation repose sur l'analyse des trames interceptées via Wireshark sur l'interface de boucle locale (*loopback*).

6.1 Analyse du Handshake TLS

Conformément à la Partie 4.2 du sujet, nous avons identifié les paquets clés de l'établissement de la connexion sécurisée sur le port 8883.

No.	Time	Source	Destination	Protocol	Length	Info
31	33.862504251	::1	::1	TCP	86	40809 → 8883 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=2943308575 TSecr=2943308575
32	33.862615932	::1	::1	TLSv1.2	295	Client Hello (SNI=localhost)
33	33.862621840	::1	::1	TCP	86	8883 → 40809 [ACK] Seq=1 Ack=210 Win=65536 Len=0 TSval=2943308575 TSecr=2943308575
34	33.863172064	::1	::1	TLSv1.2	2211	Server Hello, Certificate, Server Key Exchange, Certificate Request, Server Hello Done
35	33.863179895	::1	::1	TCP	86	40809 → 8883 [ACK] Seq=210 Ack=2126 Win=80064 Len=0 TSval=2943308576 TSecr=2943308576
36	33.865408270	::1	::1	TLSv1.2	2139	Certificate, Client Key Exchange, Certificate Verify, Change Cipher Spec, Encrypted Handshake Message
37	33.865962283	::1	::1	TLSv1.2	1186	New Session Ticket, Change Cipher Spec, Encrypted Handshake Message
38	33.866109980	::1	::1	TLSv1.2	176	Application Data
39	33.866196881	::1	::1	TLSv1.2	119	Application Data
40	33.866276244	127.0.0.1	127.0.0.1	TCP	74	34816 → 44377 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=3222888338 TSecr=0 WS=1024
41	33.866293636	127.0.0.1	127.0.0.1	TCP	74	44377 → 34816 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=3222888338 TSecr=3222888338 WS=1024
42	33.866301941	127.0.0.1	127.0.0.1	TCP	66	34816 → 44377 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3222888338 TSecr=3222888338
43	33.866808346	127.0.0.1	127.0.0.1	TCP	67	44377 → 34816 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=1 TSval=3222888338 TSecr=3222888338
44	33.866819444	127.0.0.1	127.0.0.1	TCP	66	34816 → 44377 [ACK] Seq=1 Ack=2 Win=65536 Len=0 TSval=3222888338 TSecr=3222888338
45	33.866937620	::1	::1	TLSv1.2	145	Application Data
46	33.867045656	::1	::1	TLSv1.2	119	Application Data
47	33.997117367	::1	::1	TCP	86	40809 → 8883 [ACK] Seq=2412 Ack=3266 Win=89088 Len=0 TSval=2943308620 TSecr=2943308586
48	34.992249431	127.0.0.1	127.0.2.2	DNS	86	Standard query 0x2f15 A ssl.gstatic.com OPT
49	34.992596719	127.0.2.2	127.0.0.1	DNS	117	Standard query response 0x2f15 A ssl.gstatic.com A 142.251.142.67 OPT
50	35.867506128	127.0.0.1	127.0.0.1	TCP	67	44377 → 34816 [PSH, ACK] Seq=2 Ack=1 Win=65536 Len=1 TSval=3222890339 TSecr=3222888338
51	35.867562178	127.0.0.1	127.0.0.1	TCP	66	34816 → 44377 [ACK] Seq=1 Ack=3 Win=65536 Len=0 TSval=3222890339 TSecr=3222890339
52	35.867807054	::1	::1	TLSv1.2	145	Application Data
53	35.868150725	::1	::1	TLSv1.2	119	Application Data
54	35.868185731	::1	::1	TCP	86	40809 → 8883 [ACK] Seq=2471 Ack=3299 Win=89088 Len=0 TSval=2943310581 TSecr=2943310581
55	37.868500012	127.0.0.1	127.0.0.1	TCP	67	44377 → 34816 [PSH, ACK] Seq=3 Ack=1 Win=65536 Len=1 TSval=3222892340 TSecr=3222890339
56	37.868650289	127.0.0.1	127.0.0.1	TCP	66	34816 → 44377 [ACK] Seq=1 Ack=4 Win=65536 Len=0 TSval=3222892340 TSecr=3222892340
57	37.869104909	::1	::1	TLSv1.2	145	Application Data
58	37.869505577	::1	::1	TLSv1.2	119	Application Data
59	37.869554372	::1	::1	TCP	86	40809 → 8883 [ACK] Seq=2530 Ack=3332 Win=89088 Len=0 TSval=2943312582 TSecr=2943312582
60	39.869486115	127.0.0.1	127.0.0.1	TCP	67	44377 → 34816 [PSH, ACK] Seq=4 Ack=1 Win=65536 Len=1 TSval=3222894341 TSecr=3222892340
61	39.869550246	127.0.0.1	127.0.0.1	TCP	66	34816 → 44377 [ACK] Seq=1 Ack=5 Win=65536 Len=0 TSval=3222894341 TSecr=3222894341
62	39.878008888	::1	::1	TLSv1.2	145	Application Data
63	39.878448874	::1	::1	TLSv1.2	119	Application Data
64	39.878491216	::1	::1	TCP	86	40809 → 8883 [ACK] Seq=2589 Ack=3365 Win=89088 Len=0 TSval=2943314583 TSecr=2943314583
65	41.878655456	127.0.0.1	127.0.0.1	TCP	67	44377 → 34816 [PSH, ACK] Seq=5 Ack=1 Win=65536 Len=1 TSval=3222896341 TSecr=3222894341
66	41.878098311	127.0.0.1	127.0.0.1	TCP	66	34816 → 44377 [ACK] Seq=1 Ack=6 Win=65536 Len=0 TSval=3222896342 TSecr=3222896341
67	41.878490086	::1	::1	TLSv1.2	145	Application Data
68	41.878797313	::1	::1	TLSv1.2	119	Application Data
69	41.878832260	::1	::1	TCP	86	40809 → 8883 [ACK] Seq=2648 Ack=3398 Win=89088 Len=0 TSval=2943316583 TSecr=2943316583
70	43.873416336	127.0.0.1	127.0.0.1	TCP	67	44377 → 34816 [PSH, ACK] Seq=6 Ack=1 Win=65536 Len=1 TSval=3222898345 TSecr=3222896342
71	43.873431670	127.0.0.1	127.0.0.1	TCP	66	34816 → 44377 [ACK] Seq=1 Ack=7 Win=65536 Len=0 TSval=3222898345 TSecr=3222898345
72	43.873472120	::1	::1	TLSv1.2	117	Application Data
73	43.873549496	::1	::1	TCP	86	40809 → 8883 [FIN, ACK] Seq=2679 Ack=3398 Win=89088 Len=0 TSval=2943318586 TSecr=2943316583
74	43.873576560	::1	::1	TLSv1.2	117	Encrypted Alert

FIGURE 2 – Capture Wireshark du trafic MQTT sécurisé sur le port 8883

Type de Paquet	N° Paquet	Notes et Observations
Client Hello	32	Le client propose ses versions de TLS et ses Cipher Suites.
Server Hello	34	Le Broker sélectionne **TLS 1.2** et la suite de chiffrement.
Certificate (Server)	34	Envoi de `broker.crt` pour authentifier le Broker.
Certificate (Client)	36	Spécifique au **mTLS** : le client prouve son identité via `client.crt`.
Encrypted Data	39	Les messages MQTT (PUBLISH) sont désormais totalement illisibles.

TABLE 1 – Chronologie du Handshake TLS observé dans Wireshark

6.2 Comparaison avec le Trafic non Chiffré

Une capture comparative sur le port 1883 montre que l'intégralité du *payload* MQTT (sujet et contenu du message) circule en clair. À l'inverse, sur le port 8883, Wireshark n'affiche que des segments *Application Data* chiffrés, garantissant une protection totale contre l'écoute clandestine (*eavesdropping*).

7 Réponses aux Questions d'Évaluation

- **Question 2 (Décision TLS vs mTLS) :** Le mTLS est indispensable pour le système de serrure intelligente et la flotte de camions, car l'identité de l'expéditeur est critique. Pour une station météo publique, un TLS simple suffit.
- **Question 3 (Risques des certificats auto-signés) :** L'absence de CA centralisée pour 10 000 capteurs rend la gestion des révocations impossible et expose le système aux attaques de type *Man-in-the-Middle*.
- **Question 5 (Analyse du Server Hello) :** La suite de chiffrement sélectionnée (ex : `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`) garantit le secret persistant (*Forward Secrecy*).

Conclusion Générale

Ce travail pratique a permis de démontrer la mise en œuvre opérationnelle d'une **Infrastructure à Clés Publiques (PKI)** dédiée à l'Internet des Objets. L'utilisation combinée de certificats X.509 et du protocole TLS sur Mosquitto offre une défense en profondeur : le chiffrement assure la confidentialité, tandis que l'authentification mutuelle (mTLS) verrouille l'accès au Broker aux seuls capteurs autorisés. Cette architecture constitue le socle fondamental de la confiance dans les réseaux industriels et domestiques modernes.